

Report: Optimising NYC Taxi Operations

1. Data Preparation

1.1. Loading the dataset

We load the sampled NYC Yellow Taxi dataset (`NYC_sampled_data.csv`) which was created earlier by sampling 0.8% of each hour of each day across all months.

The dataset contains approximately 303,397 rows and includes all major trip attributes such as fare amounts, timestamps, zone IDs, rate codes, passenger counts, etc.

1.1.1. Sample the data and combine the files

Before saving the sampled file, the following process was executed:

- All parquet trip files inside the folder `trip_records` were iterated.
- For each hour × day × month combination, **0.8%** of the records were sampled to keep the dataset computationally manageable.
- These sampled files were merged into one full-year dataset.
- The final sampled dataframe was saved using:

```
df_year.to_csv('/content/drive/MyDrive/IIITB_EDA_Assignment/NYC_sampled_data.csv', index=False).
```

This sampled dataset is used for all further cleaning and EDA.

2. Data Cleaning

2.1. Fixing Columns

We load the sampled dataset into `sampled_data` and inspect its structure using `.head()` and `.info()`. Since all columns contribute to temporal, spatial, or monetary analysis, **no columns are dropped** at this stage.

2.1.1. Fix the index

- A new `pickup_datetime` column is created from `tpep_pickup_datetime`.
- Converted with `errors='coerce'` to handle invalid formats.
- The dataset is sorted chronologically.
- Index is reset with `drop=True`.
- Helper columns like `pickup_datetime`, `date`, and `hour` (only used during sampling) are dropped.

This results in a clean, time-ordered dataset.

2.1.2. Combine the two airport_fee columns

The dataset included two similar columns: `airport_fee` and `Airport_fee`.

Steps taken:

1. Fill missing values in `airport_fee` using values from `Airport_fee`.
2. Drop the redundant `Airport_fee`.
Replace all remaining NaN in `airport_fee` with 0.

Outcome: A single unified, fully clean airport fee column.

2.1.3. Fix columns with negative (monetary) values

- **Identified all monetary columns** by selecting the **last 9 columns** in the dataset (these include fare amount, tip amount, total amount, taxes, surcharges, etc.).
- **Converted all these columns to numeric** using `pd.to_numeric(errors='coerce')` to ensure that:
 - Any non-numeric or corrupted values become `NaN`.
 - All values can be reliably checked for validity.
- **Checked for negative values** across all monetary amount columns since negative fares, tips, totals, or taxes are logically invalid in NYC taxi trip data.
- **Computed the count of negative values** in each monetary column using:
`negative_counts = (sampled_data[amount_cols] < 0).sum()`
- `Extra`, `mta_tax`, `improvement_surcharge`, `total_amount`, `congestion_surcharge` and `airport_fee` columns had negative values.
- Extracted all **rows containing any negative monetary value** to inspect the problematic records.
- Created a **boolean mask** to identify rows where *any* monetary column was negative using: `mask = (sampled_data[amount_cols] < 0).any(axis=1)`
- Removed all such rows from the dataset, ensuring **no monetary values are negative**, as negative fares/tips/taxes are invalid in NYC taxi operations.
- `RatecodeID -1` (Standard Rate) has the most rows with negative monetary values followed by `RatecodeID - 2` (JFK)

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

Using:

- `missing_proportion = sampled_data.isna().mean() * 100`
- `missing_columns = missing_proportion[missing_proportion > 0]`

We observe missing values mainly in:

- `congestion_surcharge`
- `passenger_count`

- `RatecodeID`
- `store_and_fwd_flag`

Each has ~3.35% missing values.

2.2.2. Handling missing values in `passenger_count`

Three-step approach:

1. **Impute NaN** with the **median passenger count = 1**.
2. **Drop invalid rows** where:
 - `passenger_count == 0` AND (`trip_distance == 0` OR `total_amount == 0`)
3. **Correct remaining zeros** by replacing `0` → `1`.

This ensures all trips have realistic passenger counts.

2.2.3. Handle missing values in `RatecodeID`

Steps implemented:

1. Remove invalid codes, e.g., `RatecodeID == 99`.
2. Rule-based imputation:
 - If `passenger_count > 1` → `RatecodeID = 6`
 - If (PU or DO zone = JFK zone 132) → `RatecodeID = 2`
 - If (PU or DO zone = Newark zone 1) → `RatecodeID = 3`
 - If zone indicates long-distance (e.g., 250) → `RatecodeID = 4`
 - If `fare_amount > 300` → `RatecodeID = 5`
 - Else fill with default = 1 (standard rate)

All missing and invalid `RatecodeID` values are resolved.

2.2.4. Impute NaN in `congestion_surcharge`

Using borough-level logic:

1. Identify all Manhattan zone IDs.
2. If `congestion_surcharge` is NaN AND (PU or DO is in Manhattan):
 - Set `congestion_surcharge = 2.50`
3. Otherwise set to 0.

This reflects NYC's congestion pricing rules accurately.

Since congestion charge is applied only in the Manhattan borough of New York city it would be fair to assign a congestion charge for rides within Manhattan and everything else 0 instead of just randomly assigning a 0 surcharge for all null values.

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

We apply multiple layers of outlier removal:

Invalid passenger count

- Remove `passenger_count > 6`

Distance–fare inconsistencies

- Extremely small distance (<0.1 miles) with very high fare (>300)
- Zero distance & zero fare but different zones
- Distance > 250 miles

Invalid payment and rate codes

- `payment_type == 0`
- `RatecodeID` not in valid range 1–6

Monetary consistency

- Negative values in any amount column
- `total_amount < fare_amount`
- `tip_amount > 2 × fare_amount`

Check allowed values

- `extra` must be in `{0, 0.5, 1, 2.5}`
- `mta_tax` ∈ `{0, 0.5}`
- `congestion_surcharge` ∈ `{0, 2.5}`

IQR-based filtering

Applied on:

- `trip_distance`
- `fare_amount`
- `total_amount`

After these steps, the dataset becomes completely clean and reliable for deeper EDA.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

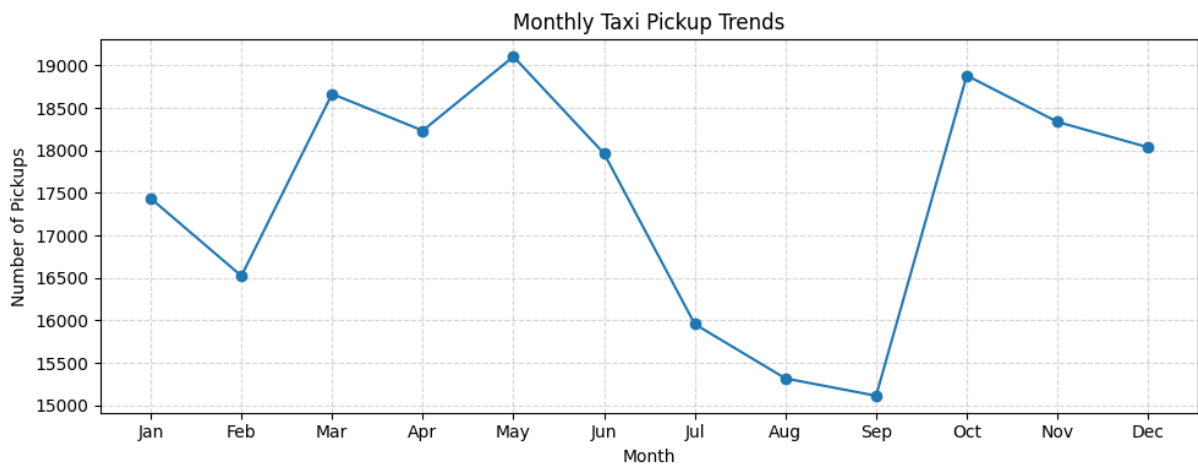
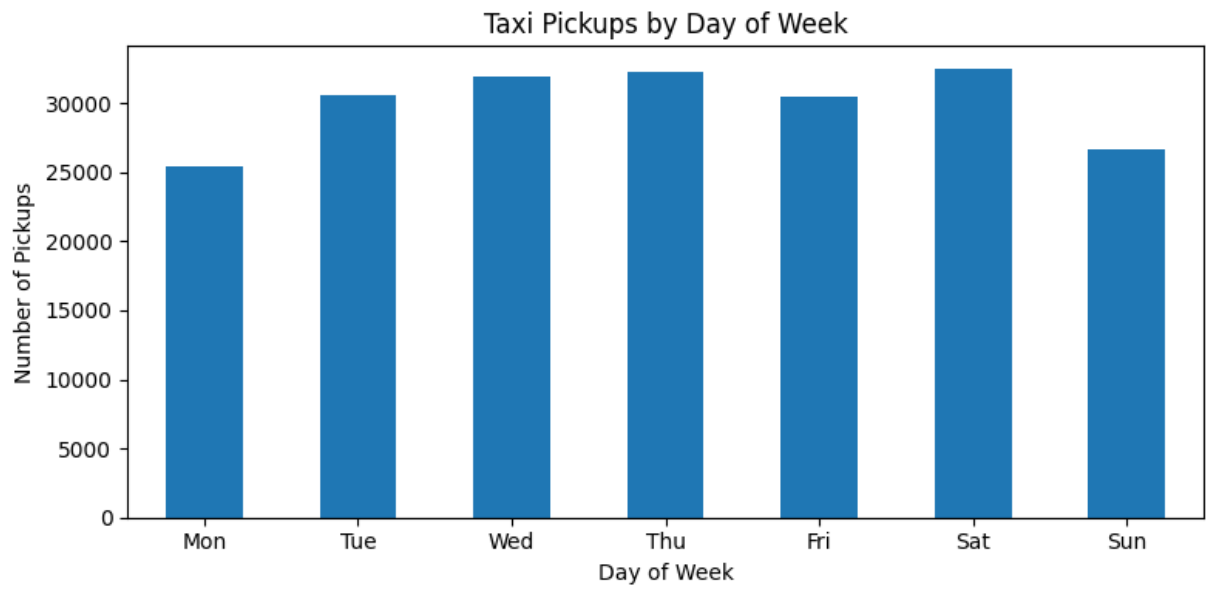
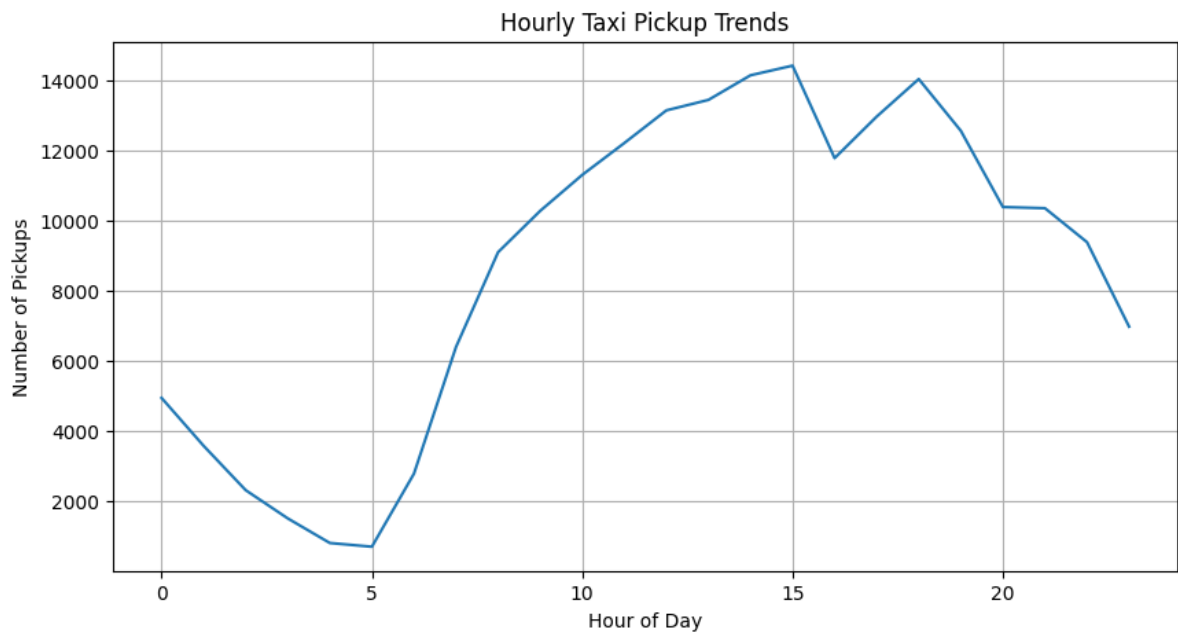
- VendorID: Categorical
- tpep_pickup_datetime: Categorical
- tpep_dropoff_datetime: Categorical
- passenger_count: Numerical
- trip_distance: Numerical
- RatecodeID: Categorical
- PULocationID: Categorical
- DOLocationID: Categorical
- payment_type: Categorical
- pickup_hour: Categorical
- trip_duration: Numerical

The following monetary parameters belong in the **numerical category**:

- fare_amount
- extra
- mta_tax
- tip_amount
- tolls_amount
- improvement_surcharge
- total_amount
- congestion_surcharge
- airport_fee

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

- Extract **pickup_hour**, **weekday**, and **month**.
- Produce hourly line chart → shows morning & evening peaks.
- Weekly bar chart → shows Friday/Saturday higher activity.
- Monthly trend → reveals seasonality across 2023.



3.1.3. Filter out the zero/negative values in fares, distance and tips

Removed:

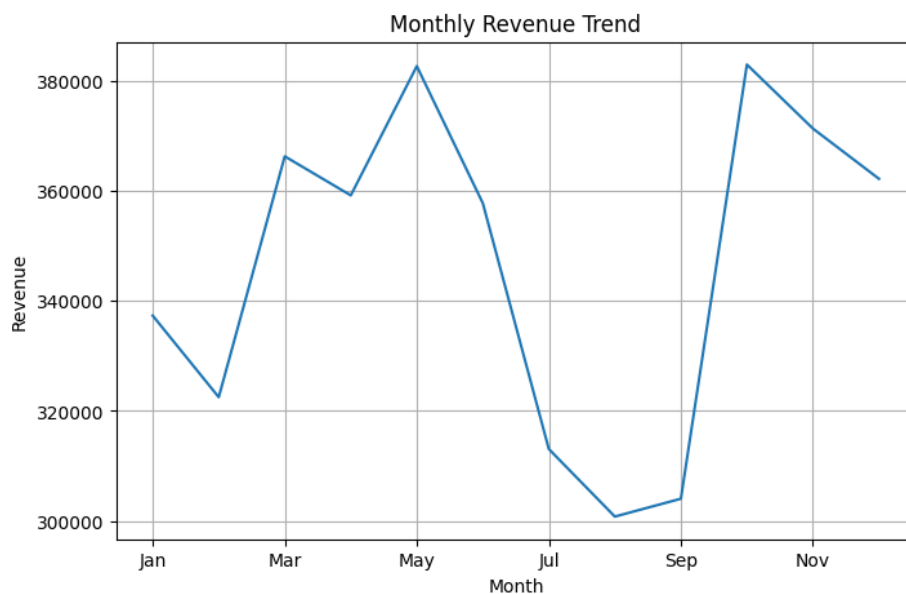
- Negative amounts
- Zero-distance but different-zone trips
- Zero-distance & zero-fare invalid trips

Kept:

- Same-zone zero-distance trips (possible meter issues or very short rides)

3.1.4. Analyse the monthly revenue trends

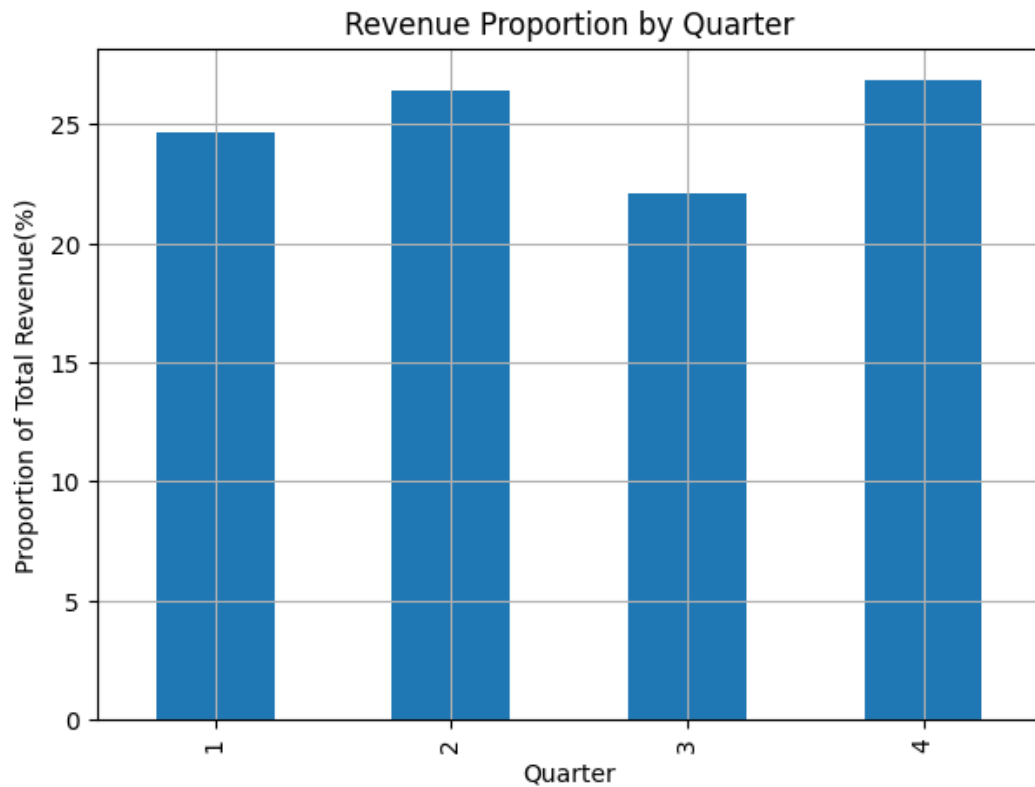
- Group by month and aggregate sum of `total_amount`
- Outcome: October generated the highest revenue followed by May.



3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

- Compute quarter = `pickup_datetime.dt.quarter`
- Group by quarter
- Compute revenue percentage share
- Outcome: Q4 generated the highest revenue(26.83%)

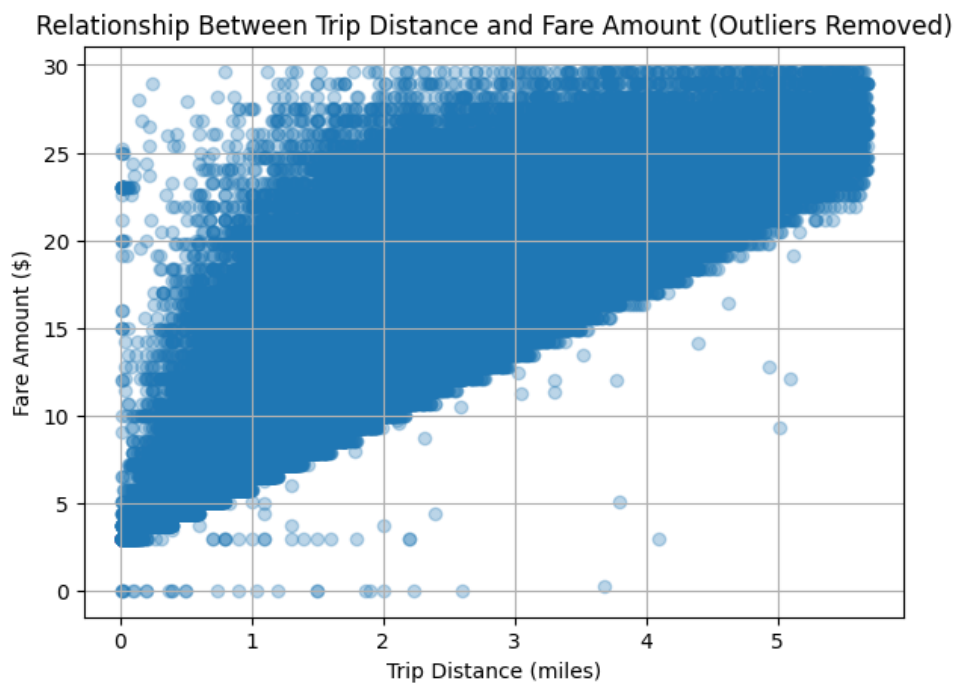
Shows how revenue concentration varies across the year.



3.1.6. Analyse and visualise the relationship between distance and fare amount

- Scatter plot: fare vs distance
- Correlation was computed using:
 - `correlation = df_nonzero['trip_distance'].corr(df_nonzero['fare_amount'])`
- Pearson correlation computed - **86.46%**

A strong positive correlation exists, confirming distance is the primary cost driver.



3.1.7. Analyse the relationship between fare/tips and trips/passengers

Correlations show:

- Fare vs duration → low correlation (0.155)
- Fare vs passenger_count → very low (0.022)
- Tip vs distance → weak to moderate (0.351)

Insight: Tip behaviour is not distance-driven; fare mainly depends on trip distance.

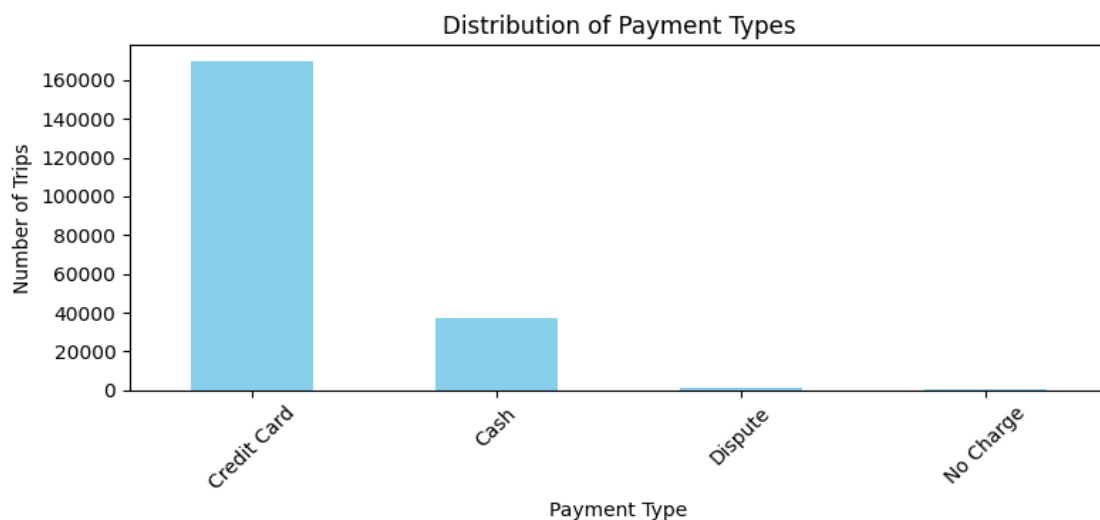
3.1.8. Analyse the distribution of different payment types

Mapped:

- 1 → Credit
- 2 → Cash
- 3 → No charge
- 4 → Dispute

Plotted bar chart.

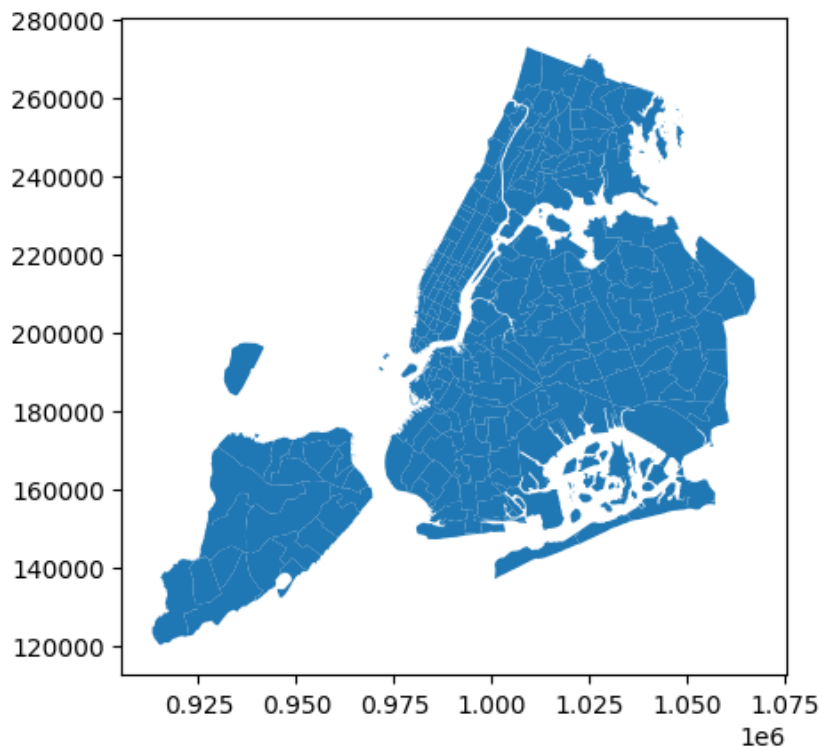
Insight: Majority(>1.6 lakh) trips are paid via credit card.



3.1.9. Load the taxi zones shapefile and display it

Shapefile `taxi_zones.shp` is loaded with GeoPandas and visualised.
All NYC taxi zones (263 polygons) appear correctly.

```
zones =  
gpd.read_file("/content/drive/MyDrive/IIITB_EDA_Assignment/Datasets/taxi_zones/taxi_zones.shp")  
zones.head()  
zones.plot()
```



3.1.10. Merge the zone data with trips data

Joined using `PULocationID` → `LocationID`, appending zone and borough names.

```
merged_pickup = sampled_data.merge(zones[['LocationID', 'zone', 'borough']],
    left_on='PULocationID',
    right_on='LocationID',
    how='left'
)
```

3.1.11. Find the number of trips for each zone/location ID

Grouped by `PULocationID` to compute total pickups.

Zones with no pickups get assigned zero.

```
trips_by_pu =
sampled_data.groupby('PULocationID').size().reset_index(name='total_pickups')
trips_by_pu.head()
```

3.1.12. Add the number of trips for each zone to the zones dataframe

Merged the aggregated pickup counts back into the shapefile GeoDataFrame.

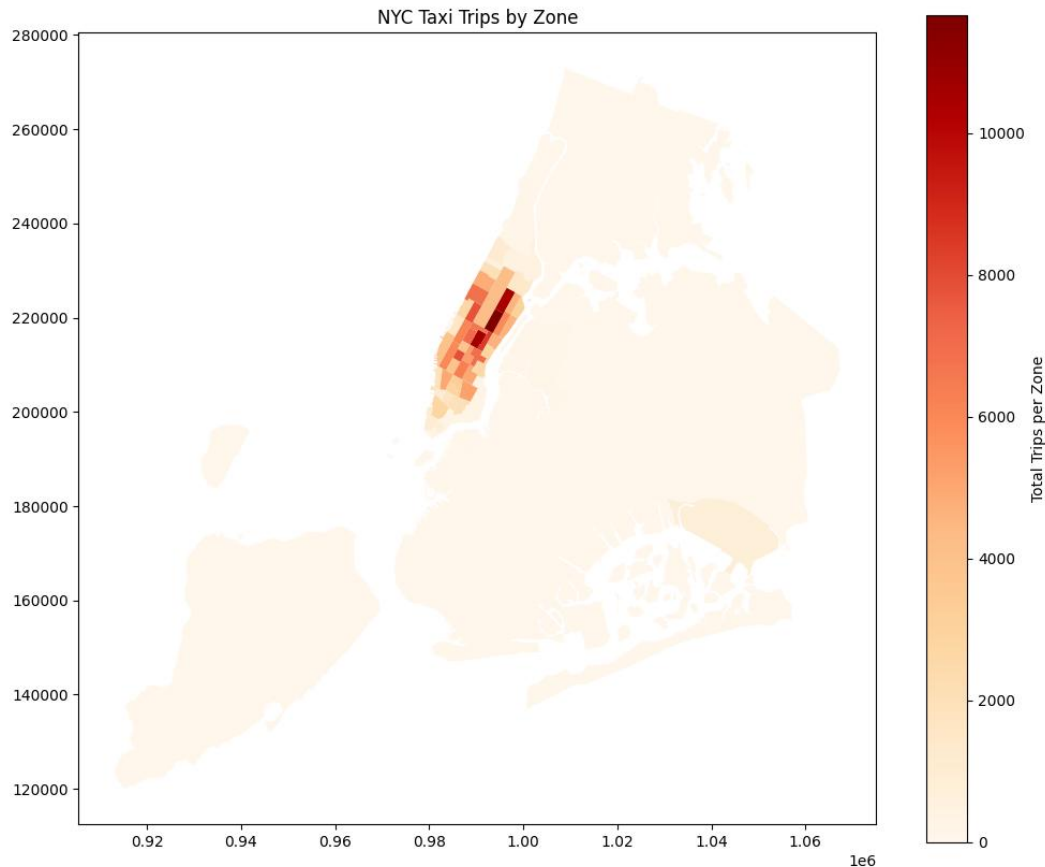
```
zones_trips = zones.merge(trips_by_pu, left_on='LocationID', right_on='PULocationID',
    how='left')
zones_trips['total_pickups'] = zones_trips['total_pickups'].fillna(0)
```

3.1.13. Plot a map of the zones showing number of trips

A choropleth map is created showing:

- Darker zones → higher taxi demand
- Lighter zones → lower demand

Insight: Demand is heavily concentrated in commercial and airport zones.



3.1.14. Conclude with results

Key Findings:

- **Trends in terms of high trip count-**
 - Busiest Hour - 3 PM
 - Busiest Day - Saturday
 - Busiest Month - May
- **Trends in revenue collected-**
 - Highest Monthly Collection - October
 - Highest Quarterly Collection - Q4
- **Fare mainly depends on trip distance** based on the correlation that we calculated.
 - Fare vs Trip Distance → high correlation (0.86)
 - Fare vs duration → low correlation (0.155)
 - Fare vs passenger_count → very low (0.022)
 - Tip vs distance → weak to moderate (0.351)
- **The top 5 busiest zones are from Manhattan-**

- Upper East Side South
- Midtown Center
- Upper East Side North
- Midtown East
- Penn Station/Madison Sq West

3.2. Detailed EDA: Insights and Strategies

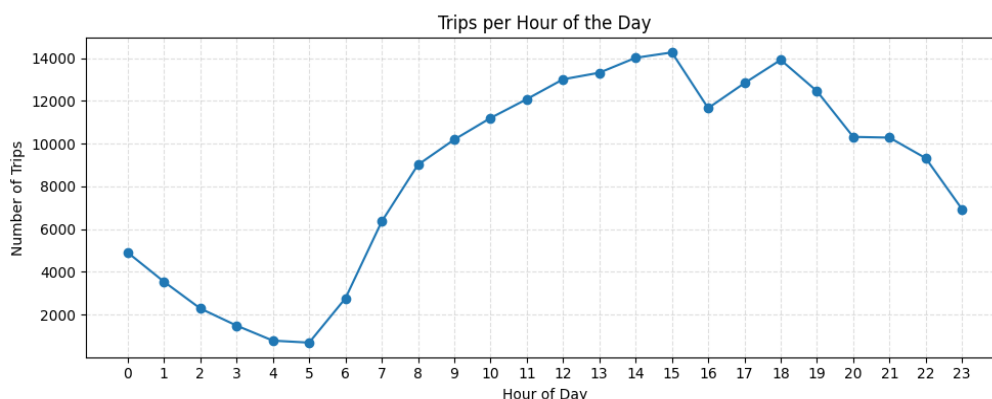
3.2.1. Identify slow routes by comparing average speeds on different routes

- Converted pickup and dropoff timestamps to datetime objects.
- Calculated trip duration in hours.
- Filtered out invalid trips (duration ≤ 0 or distance ≤ 0).
- Computed speed (mph) for each trip using:
 - `cleaned_data['trip_distance'] / cleaned_data['trip_duration_hr']`
- Grouped the data by (**PULocationID**, **DOLocationID**, **pickup_hour**) to compute the average speed for every route at every hour.
- Identified the **slowest routes per hour** by sorting speeds in ascending order and extracting the bottom five.
- These routes represent bottlenecks where congestion is highest or movement is slowest.
- Here are the top 5 slowest routes-

	PULocationID	DOLocationID	pickup_hour	speed_mph
38941	260	129	17	0.040746
27727	209	232	13	0.043579
3647	50	43	8	0.059525
7936	88	144	0	0.074923
26342	181	132	20	0.091611

3.2.2. Calculate the hourly number of trips and identify the busy hours

- Grouped trips by `pickup_hour` to get total hourly trip volumes.
- Identified the **busiest hour** using the maximum count.
- Busiest Hour: 15; Number of Trips: 14275



3.2.3. Scale up the number of trips from above to find the actual number of trips

- Since only **0.8%** of the data was sampled, applied a scaling factor of $1 / 0.008 = 125$.
- Multiplied hourly counts by 125 to estimate the **actual number of trips** in NYC.
- Extracted the top five busiest hours and their estimated total real-world trip counts.

```
# Scale up the number of trips
actual_trips_per_hour.sort_values(ascending=False).head(5)
```

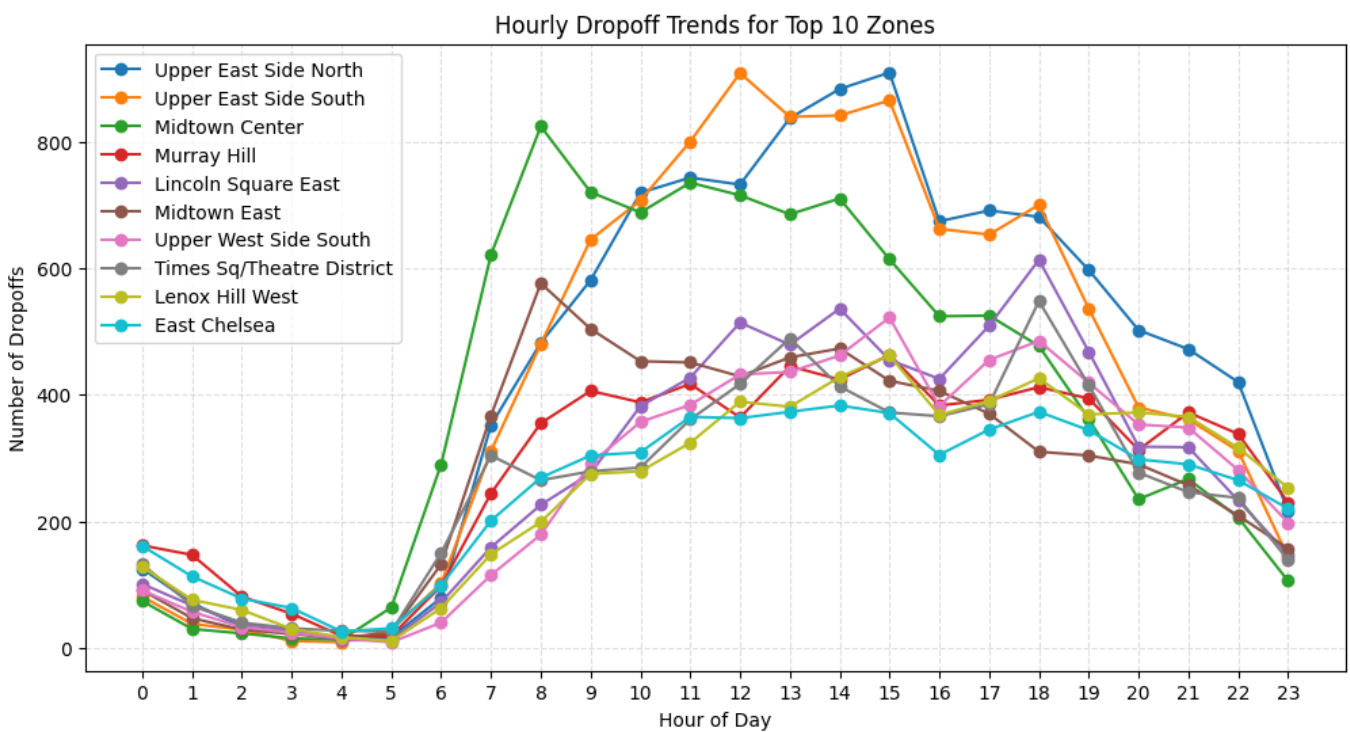
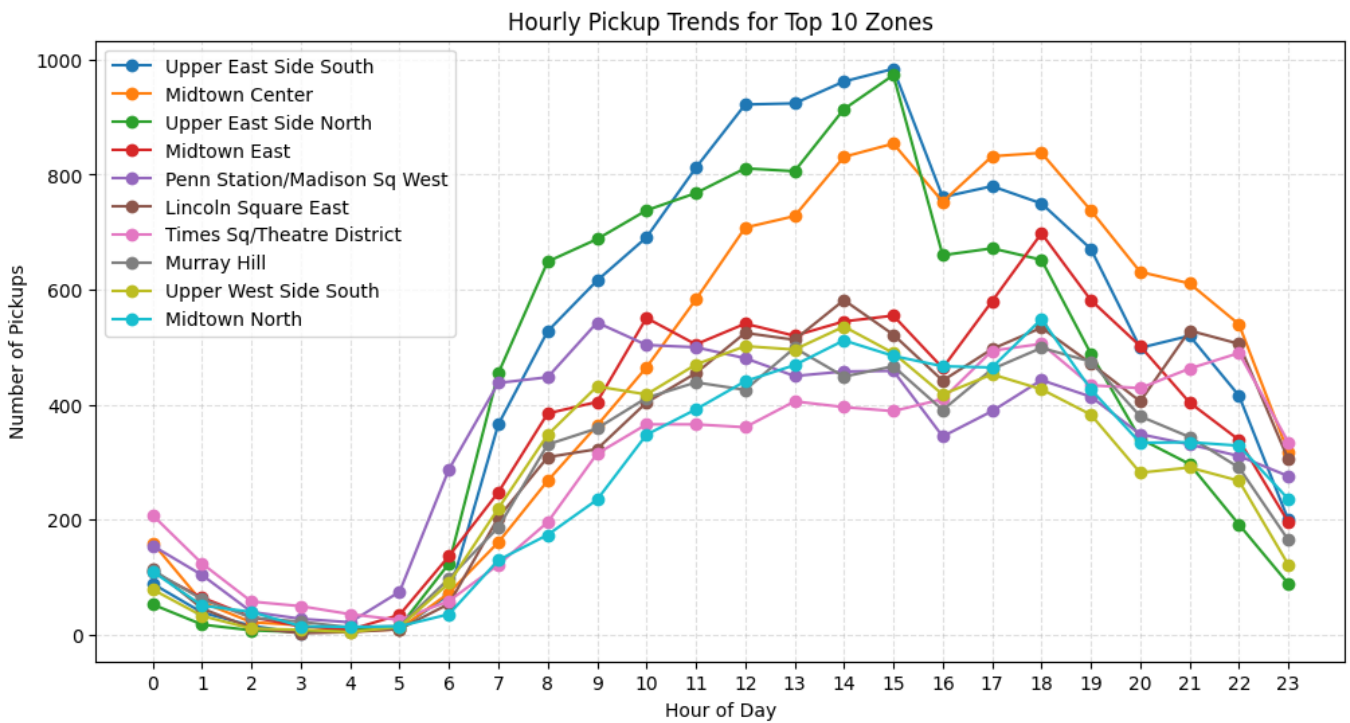
pickup_hour	0
15	1784375.0
14	1752750.0
18	1740625.0
13	1665500.0
12	1627000.0

3.2.4. Compare hourly traffic on weekdays and weekends

- Created a **day_type** column:
 - Weekday → Mon–Fri
 - Weekend → Sat–Sun
- Grouped data by **pickup_hour** and **day_type**.
- Plotted two traffic curves: Weekday vs Weekend.
- Insights show:
 - Weekdays have morning & evening commuter peaks.
 - Weekends show a gradual increase in the morning unlike a spike like weekdays.
- This helps in understanding demand differences for scheduling and surge planning.
- Overall weekdays have way more commuters than weekends

3.2.5. Identify the top 10 zones with high hourly pickups and drops

- Computed total pickups per zone and extracted the **top 10 pickup zones**.
- Computed total dropoffs per zone and extracted the **top 10 dropoff zones**.
- Mapped zones to names using the NYC taxi zones shapefile.
- Plotted hourly trends for:
 - Top 10 pickup zones
 - Top 10 dropoff zones
- These patterns highlight which neighbourhoods consistently generate or attract high demand.



3.2.6. Find the ratio of pickups and dropoffs in each zone

- Calculated total pickups and total dropoffs for each zone.
- Combined into a single dataframe and computed:
`zone_flow['ratio'] = zone_flow['pickups'] / zone_flow['dropoffs'].replace(0, 1)`
- Identified:

- Top 10 highest ratios → zones where taxis are demanded more than supplied.
 - Top 10 lowest ratios → zones where taxis arrive more than they are hailed.
- Useful for balancing cab distribution and reducing idle travel.

Top 10 Pickup/Dropoff Ratios:

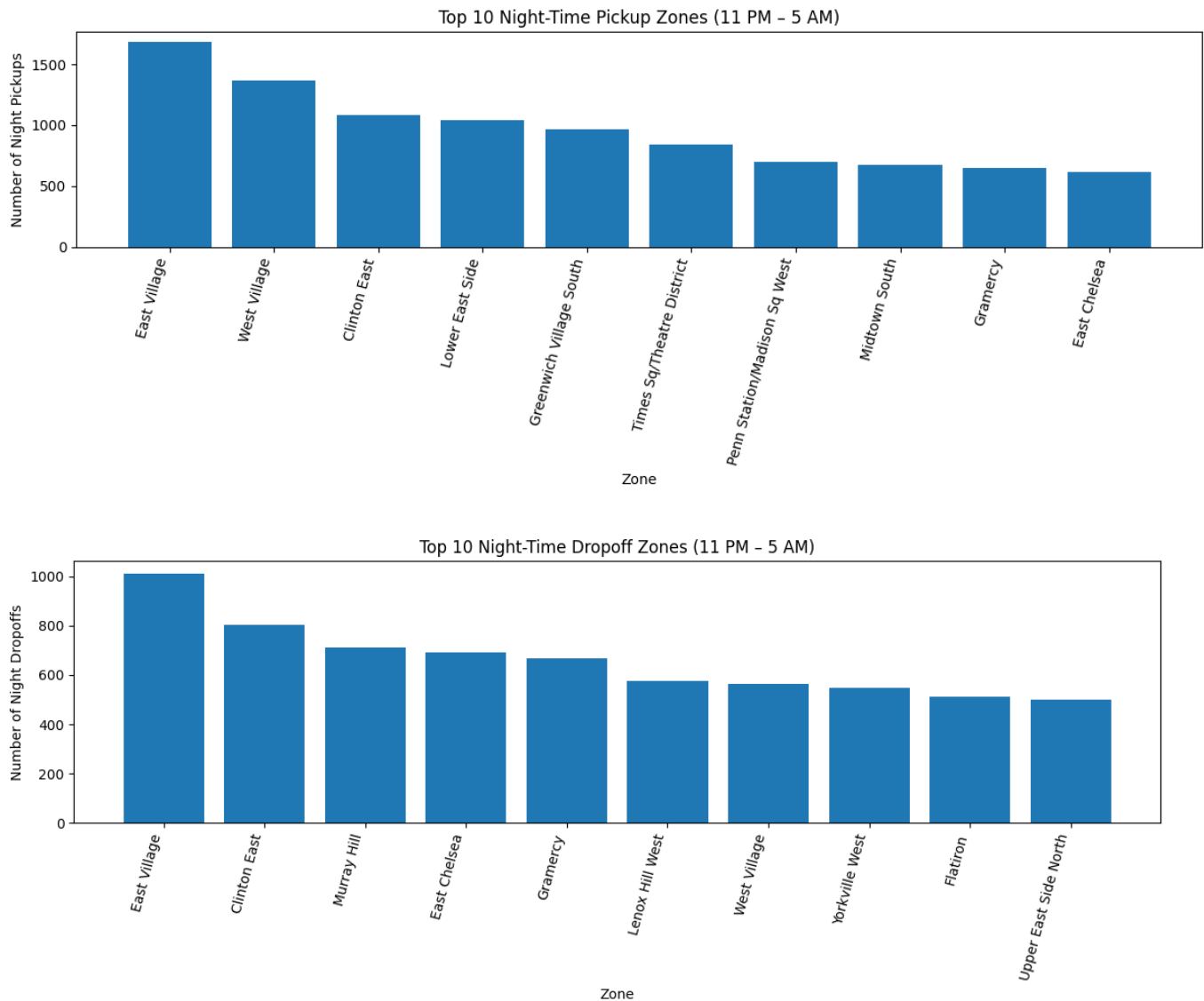
	LocationID	zone	pickups	dropoffs	ratio
96	132.0	JFK Airport	762.0	193.0	3.948187
52	70.0	East Elmhurst	53.0	16.0	3.312500
57	76.0	East New York	3.0	1.0	3.000000
139	182.0	Parkchester	3.0	0.0	3.000000
134	175.0	Oakland Gardens	3.0	1.0	3.000000
19	29.0	Brighton Beach	2.0	0.0	2.000000
43	60.0	Crotona Park East	2.0	1.0	2.000000
72	95.0	Forest Hills	17.0	11.0	1.545455
140	186.0	Penn Station/Madison Sq West	7851.0	5318.0	1.476307
201	261.0	World Trade Center	991.0	681.0	1.455213

Bottom 10 Pickup/Dropoff Ratios:

	LocationID	zone	pickups	dropoffs	ratio
77	102.0	Glendale	0.0	2.0	0.0
23	35.0	Brownsville	0.0	3.0	0.0
99	135.0	Kew Gardens Hills	0.0	2.0	0.0
26	38.0	Cambria Heights	0.0	1.0	0.0
198	257.0	Windsor Terrace	0.0	7.0	0.0
95	131.0	Jamaica Estates	0.0	3.0	0.0
90	126.0	Hunts Point	0.0	1.0	0.0
88	124.0	Howard Beach	0.0	12.0	0.0
87	122.0	Hollis	0.0	1.0	0.0
100	136.0	Kingsbridge Heights	0.0	3.0	0.0

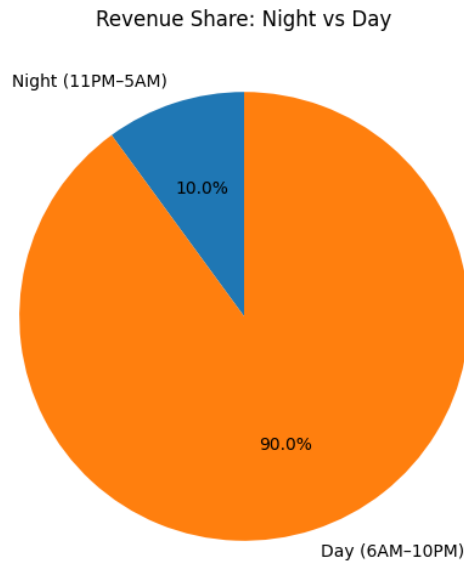
3.2.7. Identify the top zones with high traffic during night hours

- Filtered data for night hours (11 PM–5 AM).
- Computed:
 - Top 10 **night pickup zones**
 - Top 10 **night dropoff zones**
- Merged with zone names and visualised with bar charts.



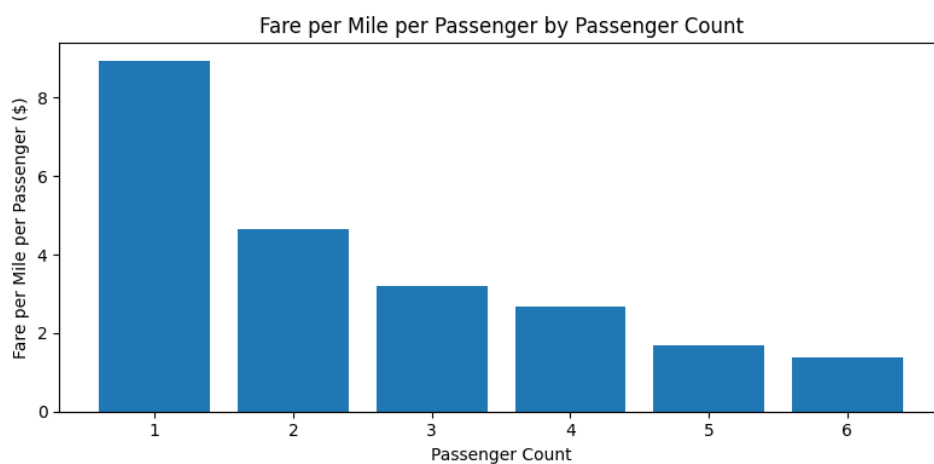
3.2.8. Find the revenue share for nighttime and daytime hours

- Divided trips into:
 - **Night:** 11 PM–5 AM
 - **Day:** 6 AM–10 PM
- Summed total revenue (total_amount) for both segments.
- Calculated percentage contribution of each.
- Visualised using a pie chart.
- This helps determine whether night pricing adjustments can be justified.



3.2.9. For the different passenger counts, find the average fare per mile per passenger

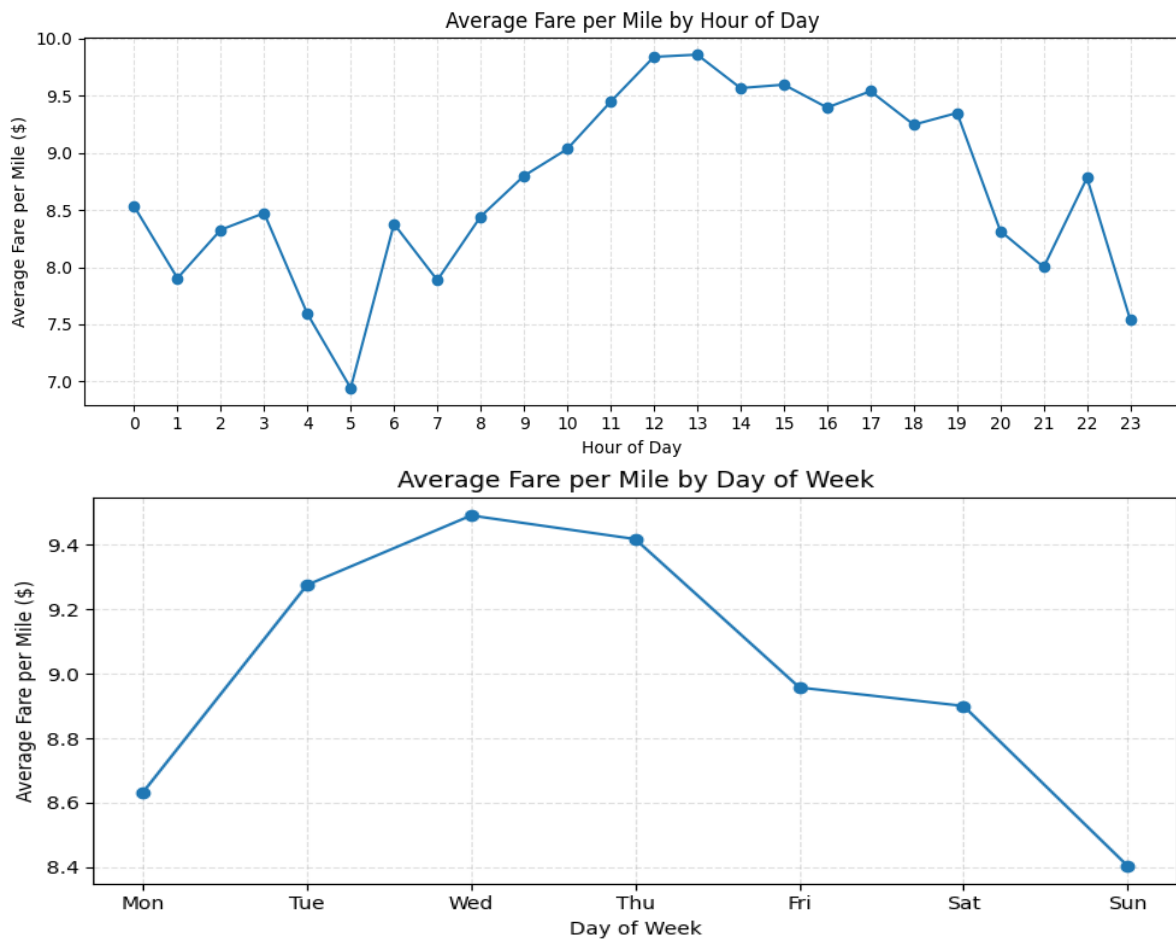
- Filtered valid trips (fare > 0, distance > 0, passenger_count > 0).
- Computed metric:
$$\text{valid['fare_per_mile_per_passenger']} = (\text{valid['fare_amount']} / (\text{valid['trip_distance']} * \text{valid['passenger_count']}))$$
- Calculated the mean fare per passenger for each passenger count.
- Helps evaluate how ride-sharing affects per-passenger cost.



3.2.10. Find the average fare per mile by hours of the day and by days of the week

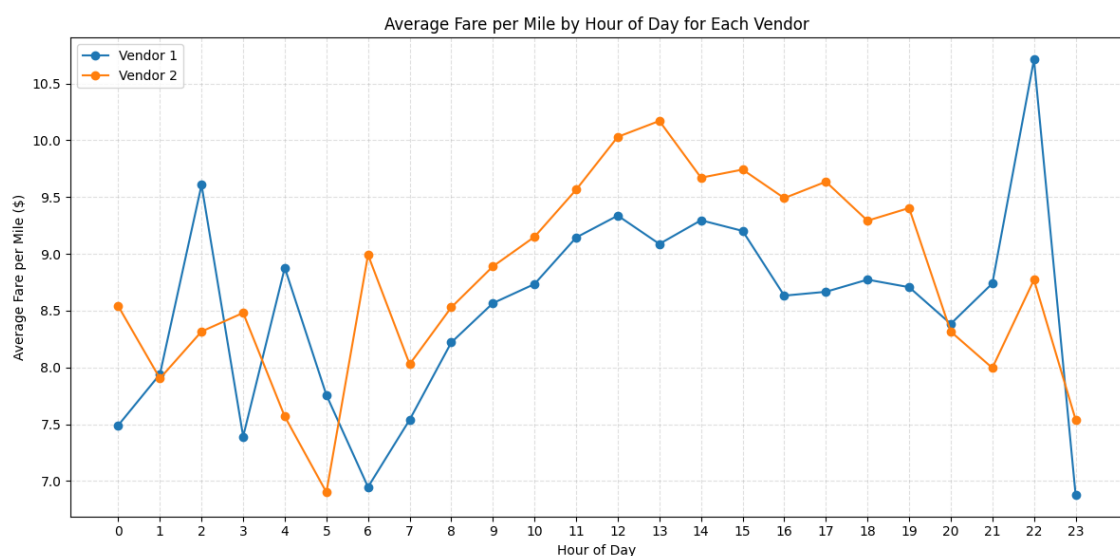
- Computed **fare_per_mile** = fare / distance.
- Grouped by:
 - **pickup_hour** → average fare per mile by hour

- `day_of_week` → average fare per mile by weekday
- Created two line plots to visualise temporal variations.
- Highlights whether pricing varies by time, congestion level, or day.



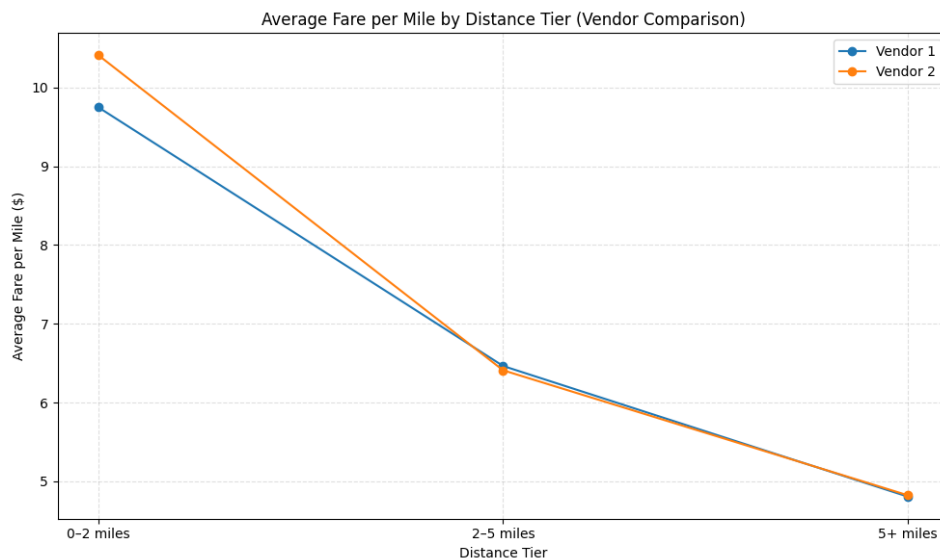
3.2.11. Analyse the average fare per mile for the different vendors

- Grouped data by (`VendorID`, `pickup_hour`).
- Computed mean fare per mile for each vendor at each hour.
- Plotted separate lines for each vendor across 24 hours.
- Identifies vendor pricing strategies or operational differences.
- **Outcome:** Vendor 2 has higher fare per mile during most of the day



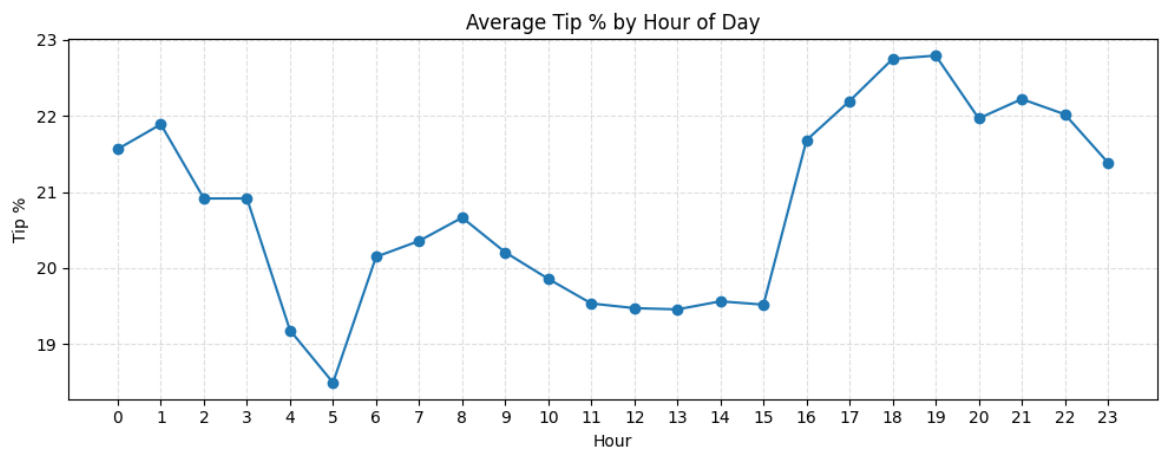
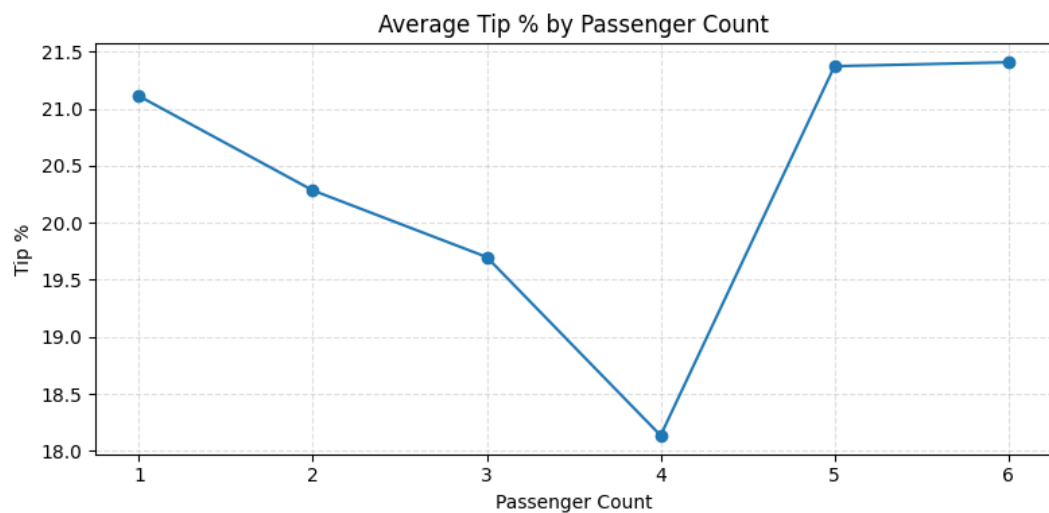
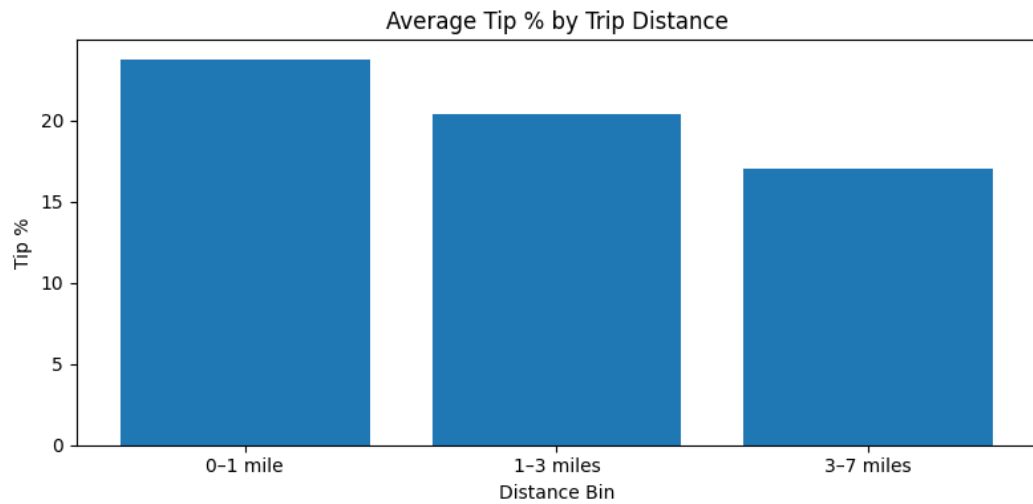
3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

- Created distance tiers: **0–2 miles, 2–5 miles and 5+ miles**
- Assigned each trip to a tier using trip_distance.
- Grouped by (**VendorID, tier**) and computed average fare per mile.
- **Outcome:** Vendors charge higher fare per mile for lower distances.



3.2.13. Analyse the tip percentages

- Computed $\text{tip_percent} = (\text{tip_amount} / \text{fare_amount}) \times 100$.
- Analysed average tip % based on:
 - **trip distance** (binned into 0–1, 1–3, 3–7, 7+ miles)
 - **passenger count**
 - **hour of pickup**
- Compared low-tip (<10%) vs high-tip (>25%) trips across:
 - fare
 - distance
 - duration
 - passenger count
 - fare per mile
- **Outcome** - Lower distance, High Passenger (5-6) and evening trips receive higher tip percentages.

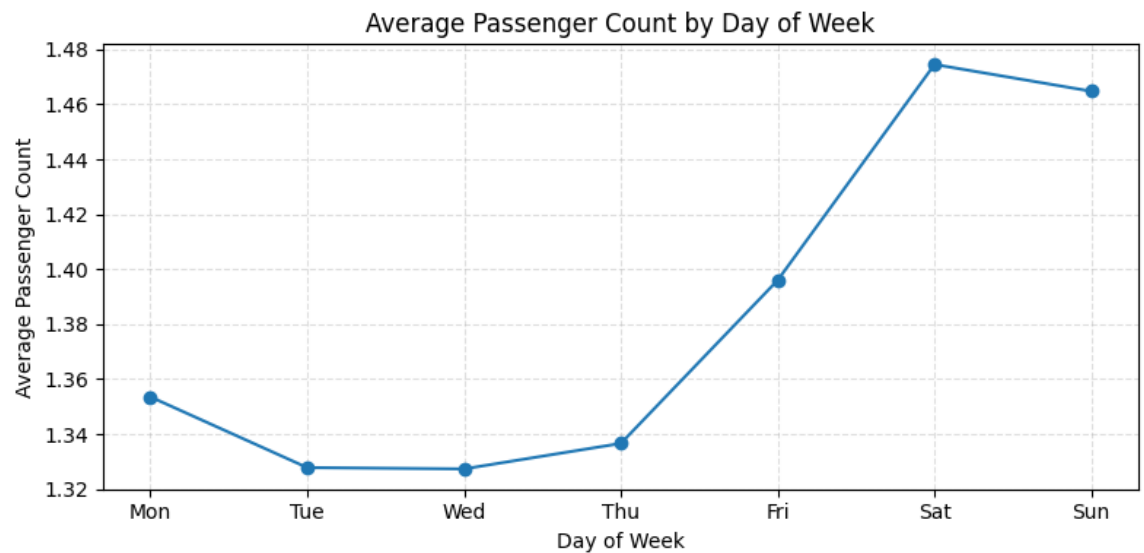
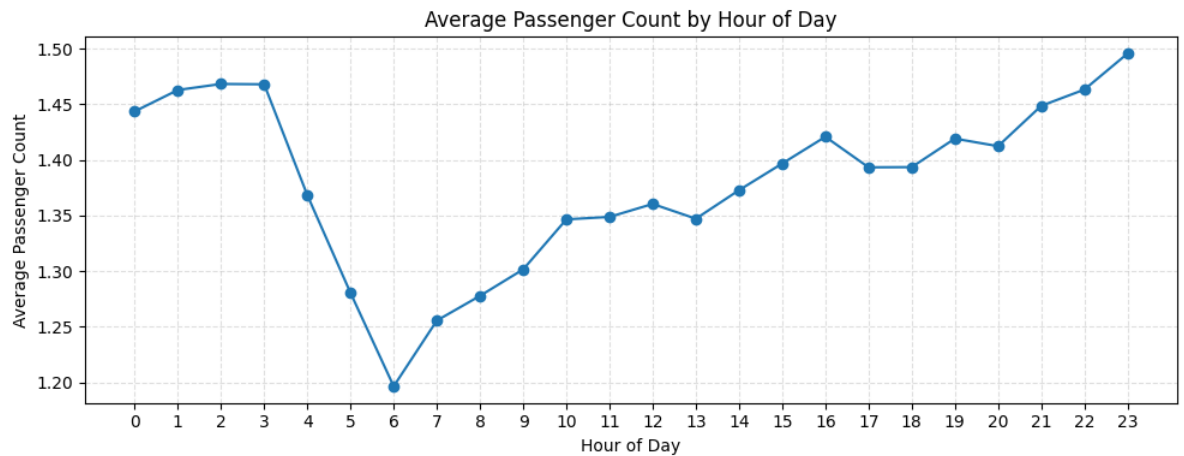


Comparing cases of low tips with cases of high tips to find out if we find a clear aspect that drives up the tipping behaviours:

	Metric	Tip < 10%	Tip > 25%
0	Average Fare (\$)	13.173660	11.208391
1	Average Trip Distance (miles)	1.781344	1.485932
2	Average Trip Duration (min)	14.902681	10.474130
3	Average Passenger Count	1.421008	1.370786
4	Average Fare per Mile (\$)	10.542828	8.681418

3.2.14. Analyse the trends in passenger count

- Grouped data by:
 - `pickup_hour` → average passengers per hour
 - `day_of_week` → average passengers per weekday
- **Outcome** - Passenger counts per ride increase at night and on weekends.

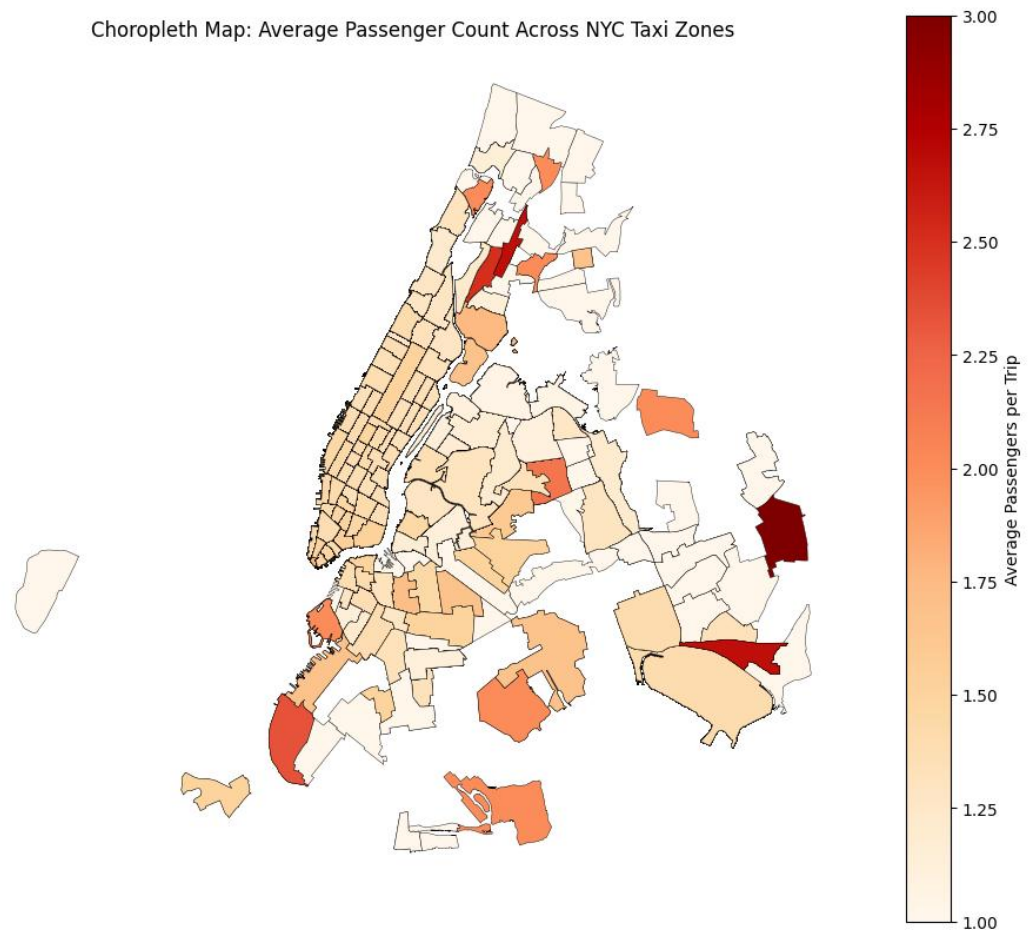


3.2.15. Analyse the variation of passenger counts across zones

- Computed average passenger count per pickup zone.
- Merged with the taxi zone shapefile.

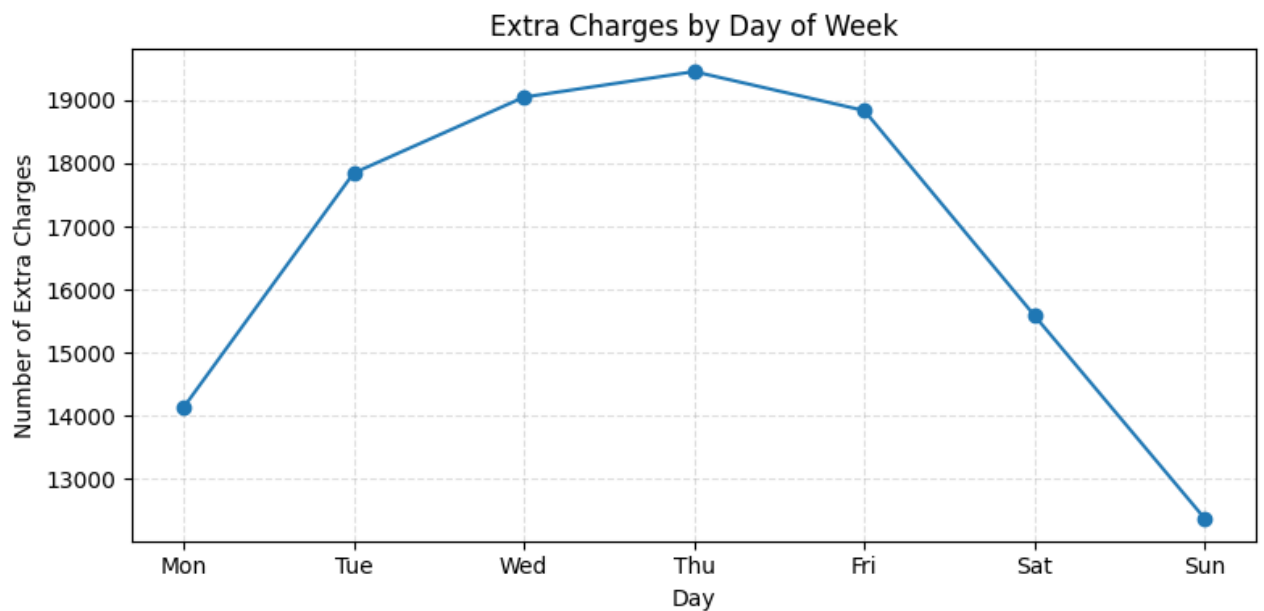
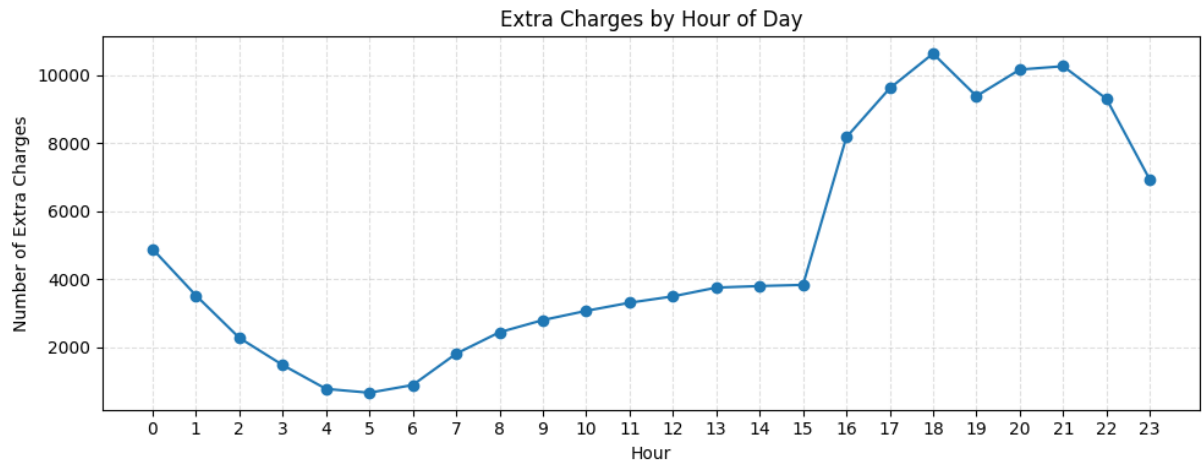
- Created a **choropleth map** showing passenger density by zone.

Choropleth Map: Average Passenger Count Across NYC Taxi Zones



3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently

- Filtered trips where **extra > 0**.
- Identified:
 - Top 10 pickup zones with highest extra charges.
 - Top 10 dropoff zones with highest extra charges.
- Outcome-** Extra charges are applied the most during the evening and late night hours, justifying the rush hour and overnight charges of \$0.50 and \$1. Thursday has the most number of extra charges.



4. Conclusion

4.1 Final Insights and Recommendations

- NYC taxi demand is **highly temporal**, with sharp hourly peaks, weekday/weekend differences, and seasonal variation.
- The **busiest window is 11 AM – 7 PM**, with over **12,000 scaled trips per hour**, indicating strong midday movement.
- **Average fare per mile is also highest during these hours**, showing that customers already expect premium pricing.

- Vendor-level analysis shows **Vendor 1 charges incorrectly**, increasing prices at night when demand actually drops.
- Spatial analysis identifies **consistent high-demand zones** for pickups/dropoffs, with nighttime hotspots being different from daytime hotspots.
- Slow-route analysis highlights **bottlenecks** in certain PU–DO combinations, especially during evening congestion hours.
- Passenger-count analysis shows **multi-passenger trips peak during specific time blocks**, useful for shared ride optimization.
- Extra charge patterns (rush-hour extras, overnight fees) show misalignments between **current surcharge windows vs. actual demand windows**.
- Overall, there is strong potential to improve taxi deployment, pricing, and routing through **dynamic, data-driven strategies**.

4.1.1 Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies

1. Routing Optimization Based on Speed and Congestion

- Use the **slowest routes per hour** to reroute taxis away from congested roads during peak times.
- Identify “bottleneck hours” where route speeds drop and recommend **alternative corridors** for drivers.
- Provide drivers with **hour-by-hour optimal route advisories** based on historical speed patterns.

2. Dispatch Optimization Based on Hourly Demand

- Increase dispatch density between **11 AM and 7 PM**, the period with maximum rides and highest fare/mile.
- Reduce idle time by prioritizing dispatch towards:
 - Major pickup clusters (e.g., Midtown, airports, downtown)
 - High-ratio pickup zones where supply < demand
- Use hourly predictability to pre-position drivers **30–45 minutes before the expected spike**.

3. Dynamic Supply Allocation

- Implement **dynamic repositioning**:
 - Move cars from low-ratio (dropoff-heavy) zones to high-ratio (pickup-heavy) zones.
 - Target zones where pickup/dropoff ratio > 1.5.
- Example: JFK Airport, East Elmhurst and East New York

4. Night-time Dispatch Strategy

- From 11 PM–5 AM focus dispatch on:

- Nightlife zones (bars, clubs, NYU area)
 - Transit hubs (Penn Station, LaGuardia)
 - Reduce oversupply in low-demand suburban zones during night hours.
 - Zones to dispatch more drivers at night-time are - East Village, West Village, Clinton East, Lower East Side and Greenwich Village South
- 5. Reduce Inefficiencies Using Speed Data**
- Avoid PU–DO pairs identified as **chronically slow** during evening peaks.
 - Encourage route diversification and optimized ETA-based dispatching.

4.1.2 Suggestions on strategically positioning cabs across different zones

1. Zone-Based Deployment Strategy

- Position higher number of taxis in **top 10 pickup zones** (identified earlier) during:
 - 11 AM–7 PM (high volume)
 - Saturday, Wednesday, Thursday (top demand days)
- Stations with consistently high dropoffs but lower pickups need **pull-in support**:
 - Example: Bring taxis into the zone after large influxes (e.g., events, dining districts).

2. Weekend vs Weekday Positioning

- **Weekdays**: Focus on business districts (Midtown, Financial District) during 8 AM–11 AM and 4 PM–7 PM.
- **Weekends**: Prioritize nightlife areas (Lower Manhattan, Brooklyn hotspots) from 9 PM–2 AM.

3. Night-time Strategic Placement

- Use the top 10 night pickup zones to place more taxis after 11 PM:
- These zones show **disproportionately high night activity** and can reduce empty mileage.

4. Passenger Count Zones

- Zones with high average passengers (e.g., tourist areas, hotel regions) should receive:
 - Larger vehicles
 - Shared-ride optimized drivers
 - Pricing tailored for multi-passenger rides

5. Balancing Pickup/Dropoff Ratios

- For **high pickup/dropoff zones**:
 - Maintain high taxi density to avoid unmet demand.

- For **low pickup/high dropoff zones**:
 - Deploy automated repositioning alerts to avoid empty return trips.
- 6. **Event-Based Micro-Zoning**
 - Periodically analyze zones with high extras/rush-hour charges:
 - These often correlate with stadium events, concerts, parades, or sudden traffic surges.

4.1.3 Propose data-driven adjustments to the pricing strategy to maximize revenue

1. **Time-Based Pricing**
 - Increase fares between **11 AM–7 PM**, especially at 3PM
 - These hours align with **peak trip counts AND highest fare-per-mile**, meaning customers already tolerate higher prices.
2. **Vendor-Level Adjustments**
 - Vendor 1 should:
 - **Increase prices during 11 AM–7 PM**
 - **Reduce unnecessary price hikes after 8 PM**, which currently do not reflect demand
 - This will align Vendor 1's pricing with true customer demand curves to retain market share while improving revenue.
3. **Day-of-Week Pricing**
 - Saturday has one of the **highest pickup volumes**, but fare-per-mile is not elevated enough.
 - Introduce **Saturday premium pricing**
 - Wednesday/Thursday also show strong demand; introduce moderate surcharges.
4. **Distance-Tiered Pricing**
 - Short trips (0–2 miles):
 - Highest congestion → highest operational costs
 - Add small surcharge (e.g., +\$0.50)
 - Medium trips (2–5 miles):
 - Maintain base pricing
 - Long trips (>5 miles):
 - Slight discount to encourage acceptance during low demand hours
 - Helps maintain competitiveness with Vendor 2
5. **Night Pricing Adjustments**
 - Reduce night premium after 12 AM unless:
 - Zone is in the top 10 night hotspots
 - Historical data shows elevated demand (e.g., weekends)
6. **Surcharge Optimization**

- Rush-hour extra charges should be applied **between 11 AM–7 PM**, not just traditional 4 PM–8 PM, because this window has:
 - Highest ride volume
 - Highest fare per mile
 - Highest congestion indicators

7. Tip-Driven Incentives

- Based on tipping analysis:
 - Shorter rides, very high passenger counts(5-6), and late-evening(6PM-7PM) trips receive more tips.
 - Encourage drivers to accept such trips.